

Asignatura	Datos del alumno	Fecha
Analítica Escalable	Apellidos:	
	Nombre:	

Actividad de refuerzo. Kafka

Trabajando con Kafka

Vamos a ver los pasos necesarios para arrancar Kafka, crear un topic y poder mandar y leer información de manera simulada. Ten en cuenta que el envío de la información (productor) lo estaría realizando una aplicación o se accedería a una fuente de datos como Twitter, por ejemplo, y el consumidor sería otra aplicación que recoge la información para procesarla. Aquí, vamos a ver cómo se simularían estos pasos.

Introducción Entorno Kafka

Bootstrap: Cada broker de Kafka es un bootstrap server, lo que significa que dicho servidor contiene un listado con todos los nodos del clúster, de manera que al conectarnos a un broker, automáticamente nos conectaremos al clúster entero.

Comandos CLI de Kafka:

[Kafka Command Line Interface \(CLI\) Tools | Confluent Documentation](#)

[Apache Kafka](#)

Pull del contenedor y run bróker Kafka

Primero vamos a hacer pull del siguiente Docker de Kafka:

[apache/kafka - Docker Image | Docker Hub](#)

Para ello entramos en línea de comandos y ejecutamos la siguiente sentencia:

docker pull apache/kafka

Asignatura	Datos del alumno	Fecha
Analítica Escalable	Apellidos:	
	Nombre:	

```
C:\Users\david>docker pull apache/kafka
Using default tag: latest
latest: Pulling from apache/kafka
e9e1a86a14df: Download complete
1673c1a2cfa7: Download complete
0b502c76566e: Download complete
634d60101085: Download complete
c4c98316ca78: Download complete
c6a83fedfae6: Download complete
c23d170f77ed: Download complete
a42500819bdc: Download complete
a39822f482a5: Download complete
e2d8f56b95eb: Download complete
Digest: sha256:c89f315cff967322c5d2021434b32271393cb193aa7ec1d43e97341924e57069
Status: Downloaded newer image for apache/kafka:latest
docker.io/apache/kafka:latest

What's next:
  View a summary of image vulnerabilities and recommendations → docker scout quickview apache/kafka

C:\Users\david>
```

Arrancamos Broker de Kafka:

`docker run -d --name broker apache/kafka:latest`

```
C:\Users\david>docker run -d --name broker apache/kafka:latest
fc3b333df22cd1d1581b0fd1bebe3fa08d26d823c65ca2094b1868c05b00208c
```

Abrimos la Shell del broker container

`docker exec --workdir /opt/kafka/bin/ -it broker sh`

```
C:\Users\david>docker exec --workdir /opt/kafka/bin/ -it broker sh
/opt/kafka/bin $ |
```

Creación de un topic

Para acceder a la documentación:

`./kafka-topics.sh`

Comando para crear un topic:

`./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test --create`

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test --create
Created topic topic-test.
```

Comando para crear un topic con particiones:

`./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test1 --create --partitions 3`

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test1 --create --partitions 3
Created topic topic-test1.
```

Comando para crear un topic con particiones y factor de replicación:

`./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test2 --create --partitions 3 --replication-factor 2`

Asignatura	Datos del alumno	Fecha
Analítica Escalable	Apellidos:	
	Nombre:	

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test2 --create --partitions 3 --replication-factor 2
Error while executing topic command : Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broker(s) are registered.
[2024-10-28 21:14:44,653] ERROR org.apache.kafka.common.errors.InvalidReplicationFactorException: Unable to replicate the partition 2 time(s): The target replication factor of 2 cannot be reached because only 1 broker(s) are registered.
(org.apache.kafka.tools.TopicCommand)
```

Error, factor de replicación mayor que número de brokers, el factor de replicación no puede ser mayor que el número de brokers.

```
./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test2 --create --partitions 3 --replication-factor 2
```

Finalmente, ha funcionado, se pueden listar los *topics* creados con el comando:

```
./kafka-topics.sh --bootstrap-server localhost:9092 --list
```

```
/opt/kafka/bin $ ./kafka-topics.sh --bootstrap-server localhost:9092 --list
topic-test
topic-test1
```

#Borrado

```
>./kafka-topics.sh --bootstrap-server localhost:9092 --topic topic-test1 --delete
```

```
>./kafka-topics.sh --bootstrap-server localhost:9092 --list
```

Creación de un productor

```
>./kafka-console-producer.sh
```

Lleva a documentacion, donde se pueden ver los parámetros obligatorios:

```
>./kafka-console-producer.sh --bootstrap-server localhost:9092 --topic test-topic
```

```
>escribir
```

```
>escribir
```

```
>escribir
```

CTRL-C para salir

Asignatura	Datos del alumno	Fecha
Analítica Escalable	Apellidos:	
	Nombre:	

```
/opt/kafka/bin $ ./kafka-console-producer.sh --bootstrap-server localhost:9092 --topic test-topic
```

```
>hola
>hola
>prueba
>^C/opt/kafka/bin $ |
```

Creación de un consumidor

En este punto, vamos a ver cómo consumir la información que se ha dejado en el *topic* de Kafka.

Como siempre para ver la documentación:

```
./kafka-console-consumer.sh
```

Ejecutamos un consumidor:

Para leer todos los msgs, y no solo los que se manden una vez se ha arrancado, se aplica el parámetro ‘--from-beginning’.

```
./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-
topic --from-beginning
```

```
/opt/kafka/bin $ ./kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic test-topic --from-beginning
prueba
hola
hola
prueba
```

El orden de los mensajes del consumidor puede no ser el mismo porque se creó con tres particiones, el orden solo se garantiza a nivel de partición, si solo tuviera una partición, se verían todos los mensajes en orden.